Individual Report — Minwook Lee (minux-lee)

Minwook Lee CS453 Team 6
KAIST

I. My Contributions

I set up the project repository and laid the groundwork that the rest of the team built on. My work falls into four areas: development environment, EvoSuite fitness implementation, project documentation, and team workflow support.

A. Development Environment and Repository

I was responsible for setting up the shared development environment end to end. This included creating the GitHub repository (minux-lee/CS453-team6), initial directory layout, and `.gitignore`; configuring the Linux server user environment used for long experiment runs; and preparing local tooling (Python `requirements.txt`, benchmark JARs under `evosuite/Benchmark_Commons/lib/`, Maven build path). I committed the skeleton EvoSuite integration in `StrongMutationSuiteFitness.java`, which defined the initial structure for operator bucketing and diversity coupling before the full implementation was completed. I later reorganised `analysis/` and `automation/` to the repository root so the pipeline structure matches the final README.

Several teammates were not familiar with Git branching. I walked them through creating branches, opening pull requests, and merging via PR review so parallel work could land without blocking the main branch. I also reviewed and merged teammate PRs (#1–#4) as changes came in.

B. EvoSuite Fitness Implementation

My main technical contribution is the diversity-aware mutation fitness in EvoSuite. In `StrongMutationSuiteFitness.java` I:

- implemented operator-type bucketing for nine mutation operators plus an "other" category;
- added `computeDiversityDeficiency()` using Shannon entropy over killed mutants;
- integrated five coupling modes into `getFitness()` (`NONE`, `MULTIPLICATIVE`, `ADDITIVE`, `EXPONENTIAL`, `CAPPED_MULTIPLICATIVE`);
- exported `MutantTypeEntropy` and `MutantTypeEntropyNorm` to EvoSuite statistics.

In `Properties.java` I added the `DiversityCoupling` enum and runtime parameters `diversity_coupling`, `diversity_k`, and `diversity_cap`. In `RuntimeVariable.java` I registered the new output variables for experiment logging.

During development I fixed seven defects in the skeleton code (null-pointer risks, NaN handling, incorrect fitness direction, and related integration issues) so the modified fitness function works correctly with EvoSuite's minimisation semantics. I also fixed a fresh-clone build issue in `Properties.java` so new contributors can build the project without extra steps.

We based the EvoSuite source on the latest upstream release from GitHub at the time of the project.

C. Documentation

I wrote and maintained the project README, including motivation, approach formulas, experimental setup, results summary, and run instructions. This documentation became the reference for the team report and presentation.

II. Peer Assessment

Each teammate is scored on a scale of 1–10 (10 = excellent). Scores exclude myself. After my EvoSuite work was in place, teammates built automation, ran experiments, and analysed results in complementary roles.

Member	GitHub	Score
Minwoo Kim	`aijk679`	10
Junho Hwang	`freeutil06`	10
Seonwoo Ji	`neutrino1359`	10

Minwoo Kim — Phase B–D automation; ran large-scale experiments. **Junho Hwang** — Core test automation and sweep runner. **Seonwoo Ji** — Result analysis, metrics, and presentation figures.